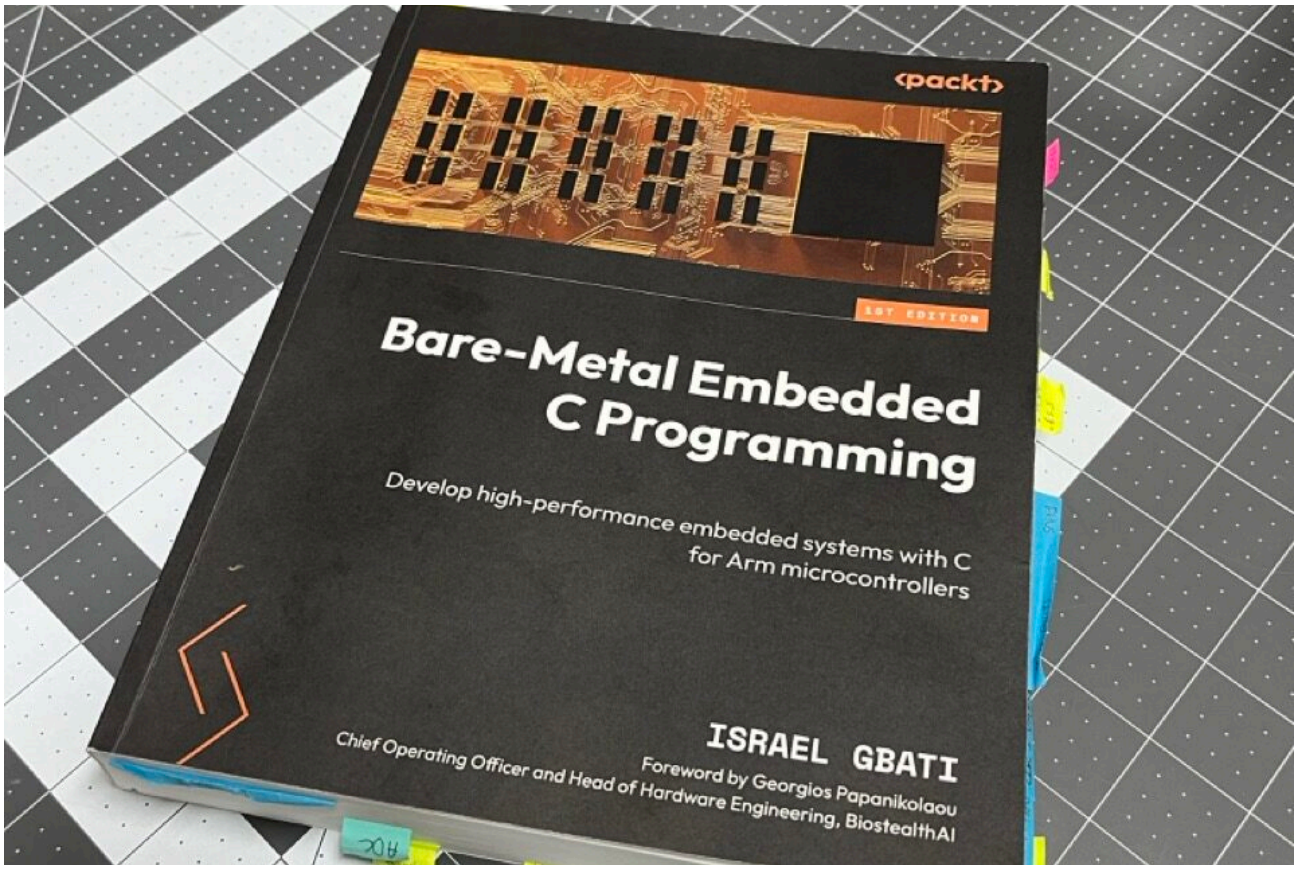
Bare-Metal Embedded C Programming: Understanding More



Written by Rob Rico in Projects, STM32, STM32CubeMX

From Register Abstractions to Silicon Fundamentals

After wrestling with the RP2040's ROM assumptions and memory bootstrapping challenges in my OTA bootloader project, I realized

something crucial: my embedded knowledge had significant gaps at the foundational level. While I could navigate Wi-Fi connectivity and JSON parsing for projects like PicoBook, the moment I needed to manually initialize runtime environments or understand what happens before `main()`, I was working with incomplete understanding.

The OTA project's failure wasn't just about RP2040 quirks or Pico SDK assumptions. It exposed a broader truth about modern embedded development: HAL libraries and frameworks like Arduino, ESP-IDF, and STM32CubeIDE create powerful abstractions, but they can mask fundamental concepts until you need to step outside their boundaries. When you're manually jumping between firmware partitions and reconstructing boot environments, those abstractions become obstacles rather than aids.

Rather than continue building complex projects on shaky foundations, I made a strategic decision: master the fundamentals through deliberate study before launching my next major initiatives. I picked up "Bare-Metal Embedded C Programming" by Israel Gbati and committed to working through it systematically with an STM32F411RE development board.

This wasn't academic exercise for its own sake. I had The Earth Rover project on the horizon, which would require sophisticated motor control using an STM32L432KC, ESP32-C3 wireless communication, and precise SPI coordination between multiple microcontrollers. Having recently struggled with low-level memory management, I wanted to approach this complexity with genuine confidence rather than educated guessing.

The learning journey proved transformative. Working directly with registers, implementing custom startup code, and understanding peripheral structures without HAL abstractions gave me the foundation to confidently use STM32CubeMX and CMake for The Earth Rover's motor control board. More importantly, the book's methodical approach to low-level programming ignited a deeper curiosity about computational fundamentals—pushing me even further down the abstraction stack to

study how CPUs operate at the silicon level and explore the evolution of digital logic itself.

What began as intentional knowledge building to understand embedded systems more deeply became a comprehensive foundation that now supports both my current Earth Rover development and my exploration of CPU-free digital logic in the DINO sequencer project. The goal was always to see where deeper understanding would lead, and it opened pathways I hadn't anticipated when I first cracked open that book.

The Learning Acceleration and Documentation Realization

Working through the first nine chapters, I found myself moving faster than anticipated. The concepts were clicking, and I was refactoring code continuously rather than maintaining separate projects per chapter. What started as careful, methodical progress became an accelerated deep dive through GPIO control, timer implementation, USART communication, and SysTick integration – covering the fundamentals that would prove essential for complex multi-MCU projects.

As I progressed through the more advanced peripherals—ADC configuration, external interrupts (EXTI), SPI communication, DMA transfers, and power management—I was exploring, implementing, and then refactoring without adequate documentation. The code worked, the concepts were understood, but the learning details were being lost in the rapid iteration.

This realization marked a critical turning point in my engineering practice. I recognized that the complexity I was planning to tackle with The Earth Rover and DINO projects would require meticulous documentation, not just for others, but for future debugging and iteration cycles.

Practical Application and Process Evolution

The bare-metal experience provided the confidence to approach STM32CubeMX as a tool I understood rather than a black box I hoped

would work. When configuring the STM32L432KC for The Earth Rover's motor control board, I could evaluate the generated code against my register-level knowledge and make informed decisions about peripheral configurations.

The accelerated learning process had taught me advanced peripheral concepts, but without proper documentation, those details were effectively lost. This gap led to a fundamental shift in my engineering practice: systematic documentation became non-negotiable. Both The Earth Rover and DINO projects now maintain detailed handwritten engineering journals, capturing every implementation challenge, design decision, and lesson learned—ensuring that hard-won knowledge doesn't disappear in the next refactoring cycle.

Transforming Engineering Approach

Beyond specific technical skills, this deep dive fundamentally changed how I approach embedded challenges. Instead of pattern matching from online examples or hoping HAL abstractions work correctly, I now have the confidence to dig into datasheets and reference manuals when encountering unfamiliar territory.

More significantly, understanding registers and peripherals at the silicon level sparked a curiosity about computational fundamentals that continues driving my learning. Questions about how CPUs themselves operate, how digital logic evolved, and what's possible outside traditional microcontroller paradigms led directly to DINO's CPU-free design exploration.

The book's methodical progression established a learning framework I now apply to every new technology: understand the fundamentals first, then build complexity systematically. Whether debugging SPI timing issues or designing address sequencing logic, the discipline of first-principles thinking has become central to my engineering approach.

Sometimes the best way to build complex systems is to first master the fundamentals that make them possible.